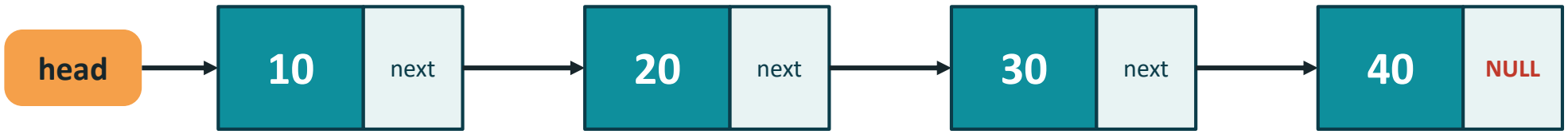
Linked List in C++

Data Structures | Simple explanation, types and a C++ program



What is a Linked List?

- A linked list is a chain of boxes. Each box is called a node.
- Every node has 2 parts: data (the value) and next (the address of the next node).
- The first node is called head. The last node points to NULL (the end).



Nodes are joined by arrows (pointers). They can be anywhere in memory.

Why Linked List? Array vs Linked List



Array

- Fixed size
- Items sit side by side in memory
- Insert / delete in the middle is slow
- Fast access: arr[3]



Linked List

- Size can grow or shrink
- Items can be anywhere in memory
- Insert / delete is easy
- Slow access: start from head

A Node in C++

1. Define the node

```
struct Node {
    int data;
    Node* next;
};
```

2. Create one node

```
Node* a = new Node();
a->data = 10;
a->next = NULL;
```

struct — Makes our own data type (the node).

***** — Pointer: it stores an address.

new — Asks memory for a new node.

-> — Reach the data through a pointer.

Types of Linked List

1

Singly Linked List

Each node has one link. You can only move forward.

A → B → C → NULL

2

Doubly Linked List

Each node has two links. You can move forward and backward.

NULL ↔ A ↔ B ↔ NULL

3

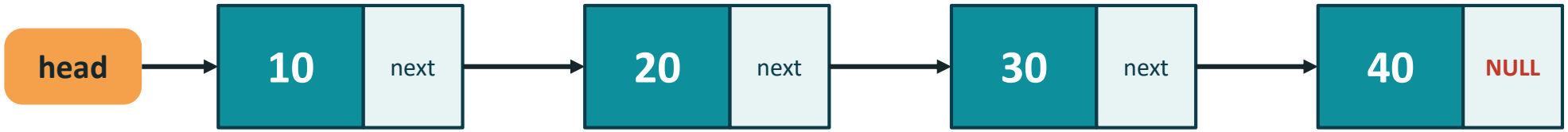
Circular Linked List

The last node points back to the first node. There is no NULL at the end.

A → B → C → back to A

Circular lists can be singly or doubly.

1. Singly Linked List



One link per node. Moves in one direction only.

How it works

Start at head. Follow next, node by node, until you reach NULL.

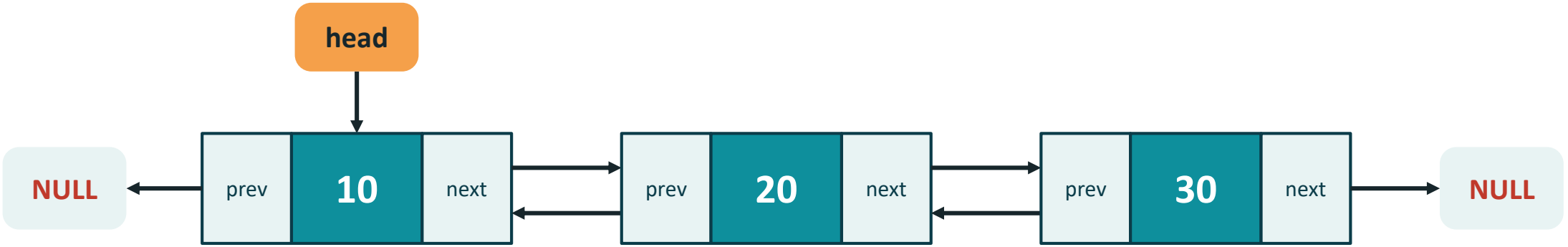
Good

Simple. Uses less memory because there is only one pointer per node.

Limit

Cannot go backward. To find a node, you must start from head.

2. Doubly Linked List



Two links per node: prev (back) and next (forward).

Node in C++

```
struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};
```

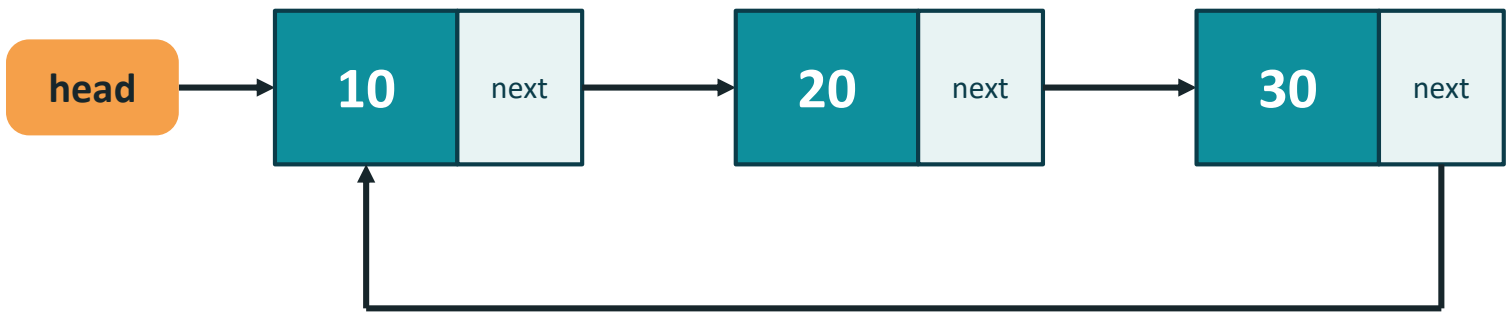
Good

You can move forward and backward. Deleting a node is easier.

Limit

Extra memory is needed for the prev pointer.

3. Circular Linked List



The last node points back to head. There is no NULL.

Good
You can go round and round. Used in music playlists and games with turns.

Careful
Stop when you reach head again, or the loop never ends.

Walk the list safely

```
Node* t = head;
do {
    cout << t->data << " ";
    t = t->next;
} while (t != head);
```

Quick Comparison

Type	Links in a node	Direction	Last node points to
Singly	1 (next)	Forward only	NULL
Doubly	2 (prev, next)	Forward and backward	NULL
Circular	1 or 2	Round and round	The first node (head)

Tip: Use Singly for simple lists, Doubly when you must go back, and Circular for things that repeat.

C++ Program - Part 1: Node and insertEnd()

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};
Node* head = NULL;

void insertEnd(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}
```

insertEnd() in 4 steps

1. Make a new node and put the value in it.
2. If the list is empty, the new node becomes head.
3. Otherwise, walk to the last node (its next is NULL).
4. Link the last node to the new node.

C++ Program - Part 2: insertStart, deleteStart, display

```
void insertStart(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void deleteStart() {
    if (head == NULL) return;
    Node* temp = head;
    head = head->next;
    delete temp;
}

void display() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

insertStart()

The new node points to the old head. Then it becomes the new head.

deleteStart()

Move head to the second node. Free the old first node.

display()

Start at head. Print each value. Stop at NULL.

C++ Program - Part 3: main() and Output

main()

```
int main() {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    insertStart(5);
    display();
    deleteStart();
    display();
    return 0;
}
```

Run it

```
g++ list.cpp -o list
```

Output

```
5 -> 10 -> 20 -> 30 -> NULL
10 -> 20 -> 30 -> NULL
```

What happened?

- insertEnd() 3 times: 10, 20, 30
- insertStart(5): 5 goes to the front
- First display() prints all 4 nodes
- deleteStart() removes 5, then display() prints 3 nodes

Remember

Node = data + next

Every box holds a value and a link.

head and NULL

Head is the start. NULL is the end.

3 types

Singly, Doubly and Circular.

Good to know

Insert and delete are easy. Search is slow.